

1. Managing Monolith Repo Main Branch Locking – Manual Control Process (Pre-Automation Phase)



Objective

To maintain CI/CD quality and production readiness by **locking the Monolith and Automation main branches** when critical regression failures occur during nightly runs.

This document outlines the **interim manual workflow**, including failure triage, communication steps, and unlock procedure — aligned with the QA automation governance at Ascensus.

Owner: Swapnil Patil

Scope: Temporary manual process to manage regression-triggered main branch locking until a fully automated solution is deployed.

Applicable Repositories

- monolith (GitLab) - For Devs
- automation (GitLab) - For QAs

Timing

- **Regression Suite Schedule:** ~1:00 AM EST (nightly run)
- **Manual Monitoring Time:** 8:00–8:30 AM EST (daily by Swapnil)

Manual Locking Workflow

Step	Description
1. Access	Swapnil has direct access to lock and unlock the main branch for both the Monolith and Automation repositories. If the root cause cannot be identified or resolved by 10:00 AM EST , the responsible Developers/Architects must revert the changes to maintain stability. <i>Note: During release weeks, extra caution should be taken. Coordinate with the release team, keep the Automation team informed, and act in alignment with deployment timelines.</i>
2. Monitor Run	At 8:00–8:30 AM, check results of the nightly regression in qTest and TestNG Report .
3. Validate Failure	Confirm failure is legit (not an environmental, DB refresh or data related issue). Look for: - Stack trace - Screenshots - Failed test name - Associated feature/module
4. Bug Documentation	Create a JIRA bug per Bug Handling SOP with: - Exception logs - Test path - Screenshot - Priority - MR link (if applicable)
5. Email Notification	Use the provided template to notify QA leads and stakeholders. Include: - Bug summary - Failure details - Impact - GitLab MR info
6. Lock Main Branch	Apply branch protection rule to lock: - monolith/main - automation/main (No MRs/commits allowed during this time)
7. Daily SLA	By 10:00 AM EST , identify the root cause of the failure. If it's a minor fix, resolve it; otherwise, revert the specific change or MR. If the responsible change cannot be identified, revert all changes made since the last successful run until the branch was locked. <i>Note: See Step 1 — apply extra caution during release weeks and coordinate with the release team.</i>
8. Manual Validation	Re-run the failed test(s) locally or in Jenkins to verify stability after fix.
9. Unlock Branch	Once stable: - Unlock main branch in GitLab - Resume pipeline
10. Send Follow-up Email	Use the resolution format per Resolution SOP . Mark bug as Resolved or Closed in JIRA.

Dry Run Testing Plan

- Simulate a regression failure intentionally.
- Practice entire end-to-end flow: 🐛 **Bug Creation** Email/Notification 🔒 **Lock** Revert / **Fix** Validate Regression TCs 🔓 **Unlock** *Follow-up Email/Notification* 🐛 **Bug Closed/Resolved**.
- Validate team readiness and timing.

Observation Phase

- Monitor this manual process for **3-4 weeks**.
- Collect data on:
 - Frequency of failures
 - Response times
 - Notification consistency
 - Stakeholder satisfaction
 - Dashboard & reporting
 - Jira Bug Team's notification

Automation Roadmap (Post-Pilot)

Once validated, implement:

- Automated detection of failed nightly regression via Jenkins/TestNG/qTest
- GitLab script to auto-lock main branch
- Auto-email to QA stakeholders
- Integration with JIRA to create or link bug ID
- Dashboard widget to show "Main Branch Status: / "

Related SOPs

- [Handling Process When a Bug Is Found or Regression Fails \(QA Automation\)](#)
- [Automation Bug Resolution: Follow-Up Process & Resolution Notification](#)

Distribution List (Manual & Automated Alerts)

- **To:**
AGSTechLeads@ascensus.com
AGSChapterLeads@ascensus.com
Brian.Danilczyk@ascensus.com
- **Cc:**
Henry.Dittmer@ascensus.com
Rajib.Akhter@ascensus.com
Phuong.Huynh@ascensus.com